

GPU Matrix Multiplication - Placement and Interview Guide

This guide develops General Matrix Multiplication (GEMM) from a direct CUDA kernel to Tensor Core and warp-level implementations. Unless stated otherwise, matrices use row-major storage and compute:

$$C[M \times N] = A[M \times K] * B[K \times N]$$

$$C[\text{row}, \text{col}] = \text{sum}(k=0..K-1) A[\text{row}, k] * B[k, \text{col}]$$

Examples are CUDA-like. Exact tile sizes are tuning choices, not universal constants.

Naive GEMM

1. Overview

Definition. A naive GEMM assigns one GPU thread to one output element of **C**. That thread walks across one row of **A** and one column of **B**, accumulating **K** products.

It is the clearest correct baseline for understanding indexing, dimensions, floating-point accumulation, and the gap between parallelism and efficiency. GEMM appears in neural-network layers, attention, graphics transforms, scientific simulation, recommendation systems, and numerical solvers. Interviewers start here because a candidate who cannot map **C[*row*, *col*]** correctly will struggle with tiled kernels.

2. Core Idea

Imagine each thread as a worker responsible for one cell in a result spreadsheet. The worker takes the matching row from **A**, the matching column from **B**, multiplies corresponding values, and adds them.

For:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$C[0,1] = 1*6 + 2*8 = 22$$

```
__global__ void gemm_naive(const float* A, const float* B, float* C,
                          int M, int N, int K) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    if (row >= M || col >= N) return;

    float sum = 0.0f;
```

```

for (int k = 0; k < K; ++k)
    sum += A[row * K + k] * B[k * N + col];
C[row * N + col] = sum;
}

```

Step by step: launch a 2D grid; derive **row** and **col**; reject excess threads; perform the dot product; write once. Each output performs about **2K** FLOPs. With no cache reuse assumed, it also requests **2K** input floats, which is the central performance problem.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Dimension compatibility	A is M x K , B is K x N , result is M x N .	$(3 \times 4)(4 \times 2) \rightarrow 3 \times 2$.	Identify the reduction dimension.
Thread mapping	One logical thread owns one (row, col) .	A 16x16 block covers 256 outputs.	Derive indices and grid dimensions.
Row-major addressing	Element (r, c) in a matrix with ld columns is base[r*ld+c] .	A[row*K+k] .	Avoid confusing mathematical dimensions and leading dimensions.
Coalescing	Adjacent lanes vary col , so they read adjacent B[k*N+col] and write adjacent C .	A warp spans neighboring columns.	Explain why A loads are broadcast-like while B loads coalesce.
Accumulation	Keep the partial sum in a register and store once.	float sum .	Never update global C on every k .
Bounds	Matrix sizes rarely match block dimensions exactly.	Guard row < M && col < N .	Correct ceiling division and safe edges.
Numerical behavior	Floating-point addition is not associative.	Different reduction order changes low bits.	Tolerance-based validation.
Leading dimensions	Submatrices and padded rows may have lda , ldb , ldc distinct from K , N , N .	A[row*lda+k] .	Production GEMM signatures.

4. Real-World Example

A fully connected neural-network layer evaluates a batch **X[B x K]** against weights **W[K x N]**, producing activations **Y[B x N]**. A naive kernel is useful as a correctness oracle for small tests, but production code calls cuBLAS because repeated input loads and weak cache control leave much of GPU throughput unused.

5. Diagrams / Mental Models

```
Thread(row=1,col=2)
```

```
A row 1:  [a10 a11 a12 a13] -- multiply pairwise --+
B col 2:  [b02 b12 b22 b32]                        |
                                                    v
C[1,2]:   a10*b02 + a11*b12 + a12*b22 + a13*b32
```

Quantity	Naive GEMM
Outputs	$M*N$
Work	$\sim 2*M*N*K$ FLOPs
Threads	Usually $M*N$
Synchronizations inside block	0
Main weakness	Redundant global-memory loads

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. What does one thread compute?	One complete dot product for one $C[\text{row}, \text{col}]$.	Ownership and no output race.	One thread per multiply.
2. Why use a 2D launch?	It maps naturally to rows and columns of C .	$x \rightarrow \text{col}, y \rightarrow \text{row}$.	Swapping dimensions inconsistently.
3. What is the time complexity?	$O(MNK)$ arithmetic overall; each output has $O(K)$ work.	Work versus parallel depth.	Claiming GPU changes algorithmic work to $O(1)$.
4. Why is the naive kernel slow?	Neighboring outputs repeatedly fetch the same rows/columns from global memory.	Low data reuse and bandwidth pressure.	Blaming multiplication latency alone.
5. Are accesses coalesced?	B and C usually are across neighboring columns; A uses the same value across those lanes, which caches/broadcast may serve.	Warp-level pattern.	Saying every load is uncoalesced.
6. Why is sum a local variable?	It normally resides in a register, avoiding K global read-modify-writes.	Register accumulation.	Calling it shared memory.
7. How is the grid sized?	$\text{ceil}(N/Bx)$ by $\text{ceil}(M/By)$.	Ceiling division and axis mapping.	Using M for x .

Question	Clear answer	Expected key points	Common mistake
8. How do rectangular matrices change the kernel?	Only dimensions and address strides change; the dot product still reduces over K .	General M, N, K .	Assuming all matrices are square.
9. How do you validate it?	Compare with a CPU/library reference using absolute plus relative tolerance.	Floating-point tolerance and varied sizes.	Exact equality for large FP32 reductions.
10. When is naive GEMM acceptable?	Teaching, correctness baselines, tiny matrices, or cases dominated by launch overhead.	Optimization must be justified.	Calling it production-optimal.

7. Deep-Dive Questions

- How many times can one input be reused?** Mathematically, each $A[i, k]$ contributes to N outputs and each $B[k, j]$ to M outputs. The naive mapping does not explicitly capture that reuse; tiling does.
- What if $K=0$?** The mathematical product is an $M \times N$ zero matrix for an overwrite-style GEMM. A generalized $C = \alpha AB + \beta C$ must still apply β .
- Why can two correct GEMMs differ numerically?** Parallel or tiled implementations change addition order and may use fused multiply-add, producing different rounding.
- What is arithmetic intensity here?** Under the pessimistic no-reuse model, roughly $2K$ FLOPs accompany $8K+4$ bytes per output, near 0.25 FLOP/byte for large K .
- What changes for column-major storage?** Address formulas and the coalesced axis change. BLAS also interprets transpose flags and leading dimensions; do not physically transpose unless useful.

8. Comparison Tables

Naive GPU GEMM	CPU triple loop	Library GEMM
One GPU thread per output	Usually sequential unless parallelized/vectorized	Hierarchical, architecture-tuned tiling
Simple and parallel	Simplest debugging reference	Highest practical performance
Redundant global loads	Cache may provide some reuse	Shared memory, registers, vector instructions/Tensor Cores
Good teaching baseline	Good small reference	Preferred production path

9. Common Mistakes

- Using $A[\text{row} * M + k]$ instead of $A[\text{row} * K + k]$.
- Launching x from M and then treating x as the column index.
- Omitting boundary checks for non-multiple dimensions.
- Accumulating directly into global memory.

- Assuming square matrices or tightly packed rows.
- Benchmarking without warm-up or synchronization.
- Comparing floating-point outputs with exact equality.

10. Edge Cases / Special Cases

- $M=0$, $N=0$, or $K=0$; avoid invalid launches and define output semantics.
- Dimensions smaller than a block or not divisible by its shape.
- Integer overflow in $\text{row} * K$; use a sufficiently wide index type for large arrays.
- Aliasing between A , B , and C is generally unsafe because writes can destroy later inputs.
- Very large or very small values can overflow, underflow, or accumulate substantial error.
- Pitched allocation and submatrix views require explicit leading dimensions.

11. How to Explain in Interview

In naive GPU GEMM, each thread owns one output element, computes the dot product of one row of A and one column of B , accumulates in a register, and writes once. It is correct and massively parallel, but it reloads input values across threads, so tiled shared-memory GEMM improves reuse and arithmetic intensity.

12. Quick Revision Notes

- Formula: $C[i,j] = \sum_k A[i,k]B[k,j]$; shapes $(M,K)(K,N) \rightarrow (M,N)$.
- Mapping: x is usually column, y is row; guard both.
- Cost: about $2MNK$ FLOPs.
- Strength: simplest correctness baseline. Weakness: redundant loads.
- Trap: coalescing and reuse are different; a coalesced load may still be repeated unnecessarily.

13. Practice Tasks

1. Implement the kernel and test 1×1 , rectangular, and non-block-multiple shapes.
2. Print indices for a 2×3 output and verify every thread's addresses by hand.
3. Compare GPU results with a CPU reference using a mixed tolerance.
4. Measure effective GFLOP/s as $2MNK / \text{time}$.
5. Add lda , ldb , and ldc , then multiply submatrix views.
6. Predict whether a 32×8 or 16×16 block gives better memory behavior, then benchmark.

14. Final Cheat Sheet

Item	Remember
Core definition	One thread computes one output dot product.
Why it matters	Establishes correct mapping and exposes redundant memory traffic.
Most asked	Indexing, dimensions, coalescing, complexity, boundary guards.
Main comparison	Naive is simple; tiled GEMM explicitly reuses inputs.

Item	Remember
One-line answer	"One thread owns one C element and loops over K , but repeated global loads limit performance."

Tiled GEMM

1. Overview

Definition. Tiled GEMM divides **A**, **B**, and **C** into smaller rectangular blocks. A thread block computes one tile of **C** by stepping through compatible tiles along the **K** dimension.

Tiling matters because matrix multiplication has enormous reuse: a small **A** tile is useful to many output columns and a **B** tile to many output rows. It is used in GPU libraries, CPU cache blocking, ML compilers, and accelerators. Interviewers ask it to test decomposition, reuse, synchronization, edge handling, and performance reasoning.

2. Core Idea

Think of moving books from a distant archive to a shared desk. Instead of each worker repeatedly walking to the archive for one page, the team brings a useful bundle once and shares it.

For a **4x4** multiplication with tile width 2, the top-left **2x2** result is:

```
C00 tile = A[rows 0..1, k 0..1] * B[k 0..1, cols 0..1]
          + A[rows 0..1, k 2..3] * B[k 2..3, cols 0..1]
```

Step by step: choose a **BM x BN** output tile; loop through **K** in chunks of **BK**; load the current **BM x BK** and **BK x BN** input tiles; multiply them into private accumulators; advance to the next **K** tile; store the completed output tile.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Interview angle
Output tile	Region of C owned by a block or warp.	BM=64, BN=64.	Map ownership without races.
K tile	Chunk of the reduction dimension.	BK=8 or 16.	Explain accumulation across phases.
Cooperative loading	Threads collectively move an input tile.	256 threads load 1024 values in rounds.	Handle loads exceeding thread count.
Data reuse	Loaded values participate in multiple FMAs.	An A value serves many C columns.	Quantify reduction in global traffic.

Subtopic	Meaning and why it matters	Example	Interview angle
Tile shape	Balances reuse, occupancy, memory transactions, and hardware limits.	Square is common, not mandatory.	No universal best tile.
Boundary tiles	Partial tiles are padded logically with zero or handled by predicates.	$M=70$ with $BM=64$.	Correctness outside clean multiples.
Hierarchical tiling	Global-to-block, block-to-warp, warp-to-thread tiling.	CTA tile contains warp tiles.	Production GEMM uses several levels.

4. Real-World Example

An inference engine multiplies a batch of activations by a weight matrix. It chooses block tiles suited to the GPU, then warp and instruction tiles beneath them. The same idea appears on CPUs: cache-blocked GEMM keeps working sets in L1/L2. Tiling is therefore a memory-hierarchy technique, not a CUDA-only trick.

5. Diagrams / Mental Models

```

      B tiles
      B0      B1
    +-----+-----+
A tile A0 | C00   | C01   |  Each C tile accumulates
    +-----+-----+  across every K tile pair.
      A1 | C10   | C11   |
    +-----+-----+

C00 = A00*B00 + A01*B10 + ... along K

```

Larger tiles provide Larger tiles cost

More reuse	More shared memory
More work per block	More registers
Better amortization	Possibly fewer resident blocks

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. What is tiling?	Partitioning matrices so a nearby-memory working set is reused before eviction.	Locality and reuse.	Describing only smaller loops.
2. Why tile the K dimension?	A K tile is a sum of products of successive A/B K tiles.	Reduction phases.	Overwriting the accumulator each phase.

Question	Clear answer	Expected key points	Common mistake
3. Must tiles be square?	No; shapes depend on matrix aspect ratio and hardware.	BM, BN, BK independently tunable.	Treating 16x16 as mandatory.
4. How does tiling reduce traffic?	Each staged A/B value is reused by several output computations.	Reuse factor.	Saying it reduces FLOP count.
5. Does tiling change complexity?	No, arithmetic remains $O(MNK)$; data movement and utilization improve.	Algorithm versus implementation.	Claiming asymptotic speedup.
6. What happens at edges?	Predicate loads/stores and supply zero for invalid reduction operands.	Partial tile correctness.	Early-return before a required barrier.
7. How do you choose tile size?	Benchmark candidates subject to threads, shared memory, registers, and occupancy.	Hardware-aware tuning.	Maximizing occupancy alone.
8. What is hierarchical tiling?	Different tile levels match global memory, shared memory, registers, and instructions.	Memory hierarchy.	One tile solves every level.
9. Why can too-large tiles hurt?	They consume resources, reduce residency, and may spill registers.	Resource tradeoffs.	More reuse is always better.
10. Is tiling useful on CPUs?	Yes, cache blocking and vector-register microkernels use the same principle.	General locality concept.	Calling it GPU-specific.

7. Deep-Dive Questions

- What is the ideal global-load reduction for a square tile of width T ?** A phase loads about $2T^2$ values and performs about $2T^3$ FLOPs, giving roughly T FLOPs per loaded float, a reuse improvement proportional to T over the naive model.
- Why might BK be smaller than BM and BN?** A shallow K tile limits shared-memory footprint and synchronization interval while keeping a broad output tile for reuse.
- What is a macro-tile versus micro-tile?** A block/CTA owns a macro-tile in C; each thread or warp owns a smaller register-resident micro-tile.
- How do skinny matrices affect tiling?** If M or N is small, large square tiles waste lanes. Rectangular tiles, split-K, or specialized kernels may work better.
- Can tiling worsen performance?** Yes: needless barriers, bank conflicts, extra address arithmetic, low occupancy, or insufficient reuse can outweigh savings.

8. Comparison Tables

Aspect	Naive GEMM	Tiled GEMM
Arithmetic work	$O(MNK)$	$O(MNK)$

Aspect	Naive GEMM	Tiled GEMM
Explicit input reuse	Little	High within a tile
Implementation	Simple	Cooperative and synchronized
Edge logic	One output guard	Predicated loads and stores
Performance ceiling	Often memory-limited	Can approach compute limit

9. Common Mistakes

- Thinking tiling reduces the number of mathematical multiplications.
- Loading tiles but still computing only one use per loaded value.
- Confusing output-tile dimensions with the K-tile depth.
- Choosing tile sizes by habit without checking resource limits.
- Failing to accumulate across all K tiles.
- Treating partial edge tiles as full valid tiles.

10. Edge Cases / Special Cases

- K smaller than BK : only one partial reduction tile exists.
- Skinny/tall matrices need non-square tile shapes.
- Batched small matrices may need one warp or several matrices per block.
- Split-K assigns multiple blocks to one output tile and needs a second reduction or atomics.
- Transposed or strided operands change which cooperative load pattern coalesces.
- Tile size must respect maximum threads, shared-memory capacity, and register allocation.

11. How to Explain in Interview

Tiled GEMM makes a thread block own a tile of C . It walks through K in chunks, cooperatively stages matching tiles of A and B , reuses them for many multiply-accumulates, and adds every phase into the same output accumulators. The arithmetic is unchanged, but global-memory traffic falls substantially.

12. Quick Revision Notes

- Tiles expose reuse; they do not change $O(MNK)$ work.
- Three independent dimensions: BM , BN , BK .
- Accumulate every K phase before storing.
- Larger tiles trade reuse for shared-memory/register pressure.
- Interview trap: boundaries require predicates, not unsafe early exits around barriers.

13. Practice Tasks

1. Draw all tile products for a 6×6 matrix using tile width 2.
2. Count naive versus tiled input loads for a clean $T \times T$ case.
3. Implement a tiled CPU GEMM and observe cache effects.
4. Benchmark 8×8 , 16×16 , and 32×8 output tiles.
5. Test $M=70$, $N=35$, $K=19$ to force partial tiles on all axes.
6. Explain how you would change the mapping for a matrix-vector product.

14. Final Cheat Sheet

Item	Remember
Core definition	Compute C in blocks and traverse K in chunks.
Why it matters	Converts global-memory reloads into local reuse.
Most asked	Tile mapping, K phases, boundaries, size tradeoffs.
Main comparison	Same FLOPs as naive, much less expensive data movement.
One-line answer	"A block accumulates one C tile from a sequence of reusable A and B tiles."

Shared-Memory GEMM

1. Overview

Definition. Shared-memory GEMM implements block tiling by copying input tiles from global memory into fast, block-visible shared memory before computing with them.

Shared memory makes reuse explicit and predictable. It is used in hand-written CUDA kernels, fused ML operators, image transforms, and as a staging layer in high-performance GEMM libraries. Interviewers use it to probe synchronization, bank conflicts, cooperative loads, resource constraints, and boundary correctness.

2. Core Idea

Shared memory is a block's scratchpad. Threads first act as movers, loading one **A** and one **B** element each. After a barrier guarantees the tile is ready, they act as calculators. A second barrier prevents fast threads from overwriting the tile while slow threads still use it.

```
template<int T>
__global__ void gemm_shared(const float* A, const float* B, float* C,
                           int M, int N, int K) {
    __shared__ float As[T][T];
    __shared__ float Bs[T][T];
    int row = blockIdx.y * T + threadIdx.y;
    int col = blockIdx.x * T + threadIdx.x;
    float sum = 0.0f;

    for (int base = 0; base < K; base += T) {
        As[threadIdx.y][threadIdx.x] =
            (row < M && base + threadIdx.x < K) ? A[row*K + base + threadIdx.x] :
0.0f;
        Bs[threadIdx.y][threadIdx.x] =
            (base + threadIdx.y < K && col < N) ? B[(base + threadIdx.y)*N + col]
: 0.0f;
        __syncthreads();

        #pragma unroll
```

```
        for (int k = 0; k < T; ++k)
            sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        __syncthreads();
    }
    if (row < M && col < N) C[row*N + col] = sum;
}
```

The zero fill makes invalid edge operands contribute nothing while every thread still reaches both barriers.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Cooperative load	Threads share the work of filling scratchpad tiles.	Each of 256 threads loads two floats.	Coalesced load mapping.
Barrier	<code>__syncthreads()</code> orders all threads in a block and makes shared writes visible.	Barrier before compute and before overwrite.	Explain both barriers.
Bank conflicts	Different addresses in the same bank serialize some shared accesses.	Transpose patterns often need padding.	<code>tile[T][T+1]</code> use case.
Broadcast	Many lanes reading the same shared address can be served efficiently on modern GPUs.	Same A value for several columns.	Do not label every same-bank access a conflict.
Static/dynamic allocation	Compile-time arrays are simple; <code>extern __shared__</code> supports runtime sizes.	Two adjacent dynamic tiles.	Size and alignment calculations.
Resource pressure	Shared bytes per block limit resident blocks per SM.	Double buffers use twice the storage.	Occupancy tradeoff.
Predication	Invalid global loads become zero; invalid stores are suppressed.	Partial M/N/K tiles.	Avoid barrier divergence.

4. Real-World Example

A fused image-processing kernel applies a learned channel transform at each pixel, effectively a small matrix multiplication. Loading an activation tile once into shared memory lets several output channels reuse it, while weights can also be staged. Fusion then applies bias and activation before the output store, avoiding another global-memory pass.

5. Diagrams / Mental Models

Global A/B (high latency)
| cooperative, coalesced loads

```

      v
+----- block -----+
| Shared As tile | Shared Bs tile |
|      \      /      |
|      thread register accumulators      |
+-----+
| one final store per output
      v
Global C

load -> barrier -> compute -> barrier -> next tile

```

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. Why use shared memory?	To load a tile once from global memory and reuse it across threads.	Explicit locality.	"It is always faster" without reuse.
2. Why is the first barrier needed?	No thread may consume a tile until all cooperative writes finish.	Visibility and ordering.	Assuming warp lockstep covers a whole block.
3. Why the second barrier?	It stops the next phase from overwriting data still being read.	Read-before-reuse hazard.	Removing it because compute ended locally.
4. Can a thread return before <code>__syncthreads()</code> ?	Not if other block threads reach the barrier; that can deadlock or be undefined.	Uniform barrier participation.	Early-returning edge threads.
5. What is a bank conflict?	Lanes request different addresses mapped to the same shared-memory bank, requiring serialization.	Address-to-bank mapping.	Confusing it with global coalescing.
6. Why pad a shared tile?	Padding changes stride so column accesses do not repeatedly map to the same banks.	Typical transpose fix.	Padding every tile without checking access pattern.
7. How are edge tiles handled?	Predicated loads write zero; all threads synchronize; valid outputs store.	Neutral element and safe barriers.	Leaving stale shared values.
8. Is shared memory a cache?	It is programmer-managed on-chip storage, though modern hardware may share physical capacity with L1.	Explicit lifetime and addressing.	Treating it as automatically populated.

Question	Clear answer	Expected key points	Common mistake
9. What limits tile size?	Threads/block, shared bytes, registers, and desired residency.	Multiple constraints.	Only checking shared capacity.
10. How do you measure improvement?	Compare correct kernels across representative shapes using GPU events and profiler traffic metrics.	End-to-end benchmarking.	Timing an asynchronous launch with a CPU clock.

7. Deep-Dive Questions

1. **When can warp synchronization replace block synchronization?** Only when all producers and consumers are within the same warp and the memory-ordering requirements are satisfied; use `__syncwarp()` where needed. Do not assume independent thread scheduling preserves old lockstep behavior.
2. **Why might shared memory not help?** If data has little reuse, fits effectively in cache, or the staging/barrier overhead dominates, explicit shared memory can be slower.
3. **How does double buffering change synchronization?** Loading and computing target different buffers; barriers or asynchronous-copy completion primitives protect buffer handoff rather than each individual instruction.
4. **What is shared-memory swizzling?** It permutes logical-to-physical addresses to avoid bank conflicts for complex warp-level access patterns, especially Tensor Core layouts.
5. **How do you distinguish a bank conflict from low occupancy?** Use profiler metrics: shared transaction/replay measures expose conflicts, while launch/resource reports expose resident warps and limiting resources.

8. Comparison Tables

Property	Global memory	Shared memory	Registers
Scope	Device	Thread block	Thread
Managed by	Hardware/program	Programmer	Compiler/thread
Typical role in GEMM	Source/destination	Reusable block tiles	Accumulators and micro-tiles
Capacity	Largest	Moderate per SM	Smallest per thread
Synchronization	Kernel/atomics/fences	Block barriers	Not shared directly

9. Common Mistakes

- Returning edge threads before a block-wide barrier.
- Omitting the barrier before a buffer is reused.
- Failing to zero invalid K-tail loads.
- Assuming adjacent global access guarantees conflict-free shared access.
- Allocating so much shared memory that only one block resides per SM.
- Using shared memory when each value is consumed once.

- Forgetting that shared memory is uninitialized at block start.

10. Edge Cases / Special Cases

- A tile partially outside M , N , or K must still participate in synchronization.
- Shared memory cannot communicate between blocks in an ordinary kernel.
- Dynamic shared-memory sections need correct offsets and alignment.
- Bank width and banking details vary by architecture; profile rather than rely on folklore.
- Very small matrices may spend more time on staging and barriers than arithmetic.
- Different transpose combinations need different global-load and shared layouts.

11. How to Explain in Interview

In shared-memory GEMM, threads cooperatively load an A tile and a B tile into a block-local scratchpad, synchronize, reuse those values for multiple FMAs, synchronize before overwriting, and repeat over K. Predicated zero loads handle edges without letting any thread skip a required barrier.

12. Quick Revision Notes

- Shared memory is explicit block-local scratchpad, not automatic cache.
- Phase order: load, barrier, compute, barrier.
- Zero-fill invalid K inputs; predicate output stores.
- Check coalescing globally and bank conflicts locally.
- Trap: divergent participation in `__syncthreads()` is unsafe.

13. Practice Tasks

1. Implement the shown kernel and compare it with naive GEMM.
2. Remove each barrier separately and explain the possible race.
3. Test partial sizes such as $31 \times 37 \times 19$ under a memory checker.
4. Use a profiler to inspect global load efficiency and shared bank conflicts.
5. Add padding to a deliberately transposed shared tile and measure the difference.
6. Calculate shared bytes/block and theoretical resident blocks for several tile sizes.

14. Final Cheat Sheet

Item	Remember
Core definition	Stage reusable A/B tiles in block-local memory.
Why it matters	Fewer global loads and controlled locality.
Most asked	Two barriers, edge predicates, bank conflicts, occupancy.
Main comparison	Shared is block-visible; registers are thread-private.
One-line answer	"Cooperative loads plus synchronization turn global traffic into many on-chip reuses."

Register Tiling

1. Overview

Definition. Register tiling makes each thread compute a small tile of output values rather than only one value. Those partial sums stay in the thread's registers.

This creates another reuse level: one loaded **A** fragment can update several columns, and one **B** fragment can update several rows. Production GEMM microkernels use it to raise instruction throughput and reduce shared-memory traffic. Interviewers ask about it to test register pressure, instruction-level parallelism (ILP), occupancy, and thread-level work mapping.

2. Core Idea

Instead of assigning a cashier one receipt cell, give that cashier a small **TM** x **TN** grid. For each **k**, the cashier reads **TM** A values and **TN** B values, then forms their outer product to update **TM*TN** accumulators.

For **TM=2**, **TN=2**:

```
float acc[2][2] = {};
for (int k = 0; k < BK; ++k) {
    float a0 = As[row0][k], a1 = As[row1][k];
    float b0 = Bs[k][col0], b1 = Bs[k][col1];
    acc[0][0] += a0*b0;  acc[0][1] += a0*b1;
    acc[1][0] += a1*b0;  acc[1][1] += a1*b1;
}
```

Four output updates use only four scalar shared loads, rather than independently loading two operands for every update. Independent accumulators also give the scheduler several FMAs to issue while earlier ones are in flight.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Thread micro-tile	TM x TN output region privately owned by one thread.	4x4 gives 16 accumulators.	Map thread IDs to rows/columns.
Outer-product update	A vector times a B vector updates the accumulator grid.	TM+TN loads feed TM*TN FMAs.	Explain register reuse.
Register pressure	Accumulators, operands, and addresses consume registers.	Larger micro-tiles may spill.	Performance tradeoff.
Spilling	Excess live values move to local memory, which resides in device memory.	Profiler shows local loads/stores.	Registers are not unlimited.
ILP	Independent accumulator chains hide FMA latency.	Update several acc[i][j] .	Complement to thread-level parallelism.

Subtopic	Meaning and why it matters	Example	Common interview angle
Occupancy	More registers per thread can reduce resident warps.	128 threads x many registers.	Maximum occupancy is not always fastest.
Store pattern	A thread's tile layout should still yield coalesced warp stores.	Lane mapping interleaves columns.	Private contiguity alone is insufficient.

4. Real-World Example

In an ML inference GEMM, a block stages activation and weight tiles in shared memory. Each thread then computes, for example, an **8x4** output micro-tile in registers. Bias and activation may be applied to those register values before coalesced stores. This avoids writing an intermediate tensor and exploits the same accumulators for fused epilogues.

5. Diagrams / Mental Models

At one k step:

```
A fragment      B fragment
[a0]             [b0 b1 b2]
[a1]             outer product
```

```
accumulator tile
[a0*b0 a0*b1 a0*b2]
[a1*b0 a1*b1 a1*b2]
```

Registers are the innermost tile in:
global tile -> shared block tile -> register thread tile

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. What is register tiling?	One thread owns multiple output accumulators and reuses operands among them.	Micro-tile and reuse.	Calling loop unrolling alone register tiling.
2. Why is it faster?	It reduces shared loads per FMA and increases ILP.	Outer-product reuse.	Saying registers reduce global loads by themselves.
3. What is its main cost?	More registers/thread can lower occupancy or spill.	Resource tradeoff.	Assuming registers have no capacity limit.
4. What is spilling?	Compiler-allocated local memory backs values that do not fit in	Local memory is off-chip/cache-	Believing local memory is on-chip because it is

Question	Clear answer	Expected key points	Common mistake
	registers.	backed.	private.
5. How does a TM $\times$ TN tile update?	Load TM A scalars and TN B scalars, then perform TM*TN FMAs.	Outer-product formulation.	Reloading for every accumulator.
6. Why does ILP matter?	Independent instructions help cover pipeline latency within a thread.	Multiple accumulator chains.	Equating ILP with occupancy.
7. Should the micro-tile be as large as possible?	No; benchmark before register pressure, spills, and reduced warps dominate.	Balanced tuning.	Monotonic-performance assumption.
8. How are results stored?	Map lanes so output stores are coalesced, predicate boundary elements.	Warp-level layout.	Each thread writing a contiguous tile without considering neighbors.
9. Does an array always live in registers?	Only if the compiler can scalarize it and indices are predictable; dynamic indexing may cause local memory.	Compile-time sizes/unrolling.	Treating C++ local arrays as guaranteed registers.
10. Register tiling versus shared tiling?	Shared tiling reuses across a block; register tiling reuses within a thread. They are usually combined.	Hierarchical reuse.	Presenting them as alternatives.

7. Deep-Dive Questions

1. **Why are compile-time tile sizes helpful?** They enable loop unrolling and scalar replacement, making register allocation and instruction scheduling easier.
2. **How can fewer resident warps still be faster?** A larger micro-tile may deliver more reuse and ILP, so each warp does more useful arithmetic and needs less shared bandwidth.
3. **What creates accumulator dependency latency?** Repeated FMAs into one accumulator form a dependency chain. Cycling through many independent accumulators lets the hardware issue other FMAs while one chain waits.
4. **How would you diagnose spills?** Inspect compiler register/local-memory reports and profiler local load/store metrics; then reduce tile size or live ranges.
5. **Why can store layout become difficult?** The compute-friendly register ownership may scatter each lane's outputs. Libraries design lane layouts or use a shared-memory epilogue to produce coalesced global stores.

8. Comparison Tables

Aspect	One output/thread	Register-tiled thread
Accumulators	1	TM*TN

Aspect	One output/thread	Register-tiled thread
Operand reuse inside thread	Low	High
ILP	Limited	Higher
Register demand	Low	Higher
Mapping/store complexity	Low	Higher
Spill risk	Small	Must be checked

9. Common Mistakes

- Making the micro-tile large enough to spill.
- Assuming a local array is necessarily register-resident.
- Ignoring coalescing when mapping micro-tile stores.
- Confusing register reuse with inter-thread sharing.
- Maximizing occupancy at the expense of useful ILP and reuse.
- Updating accumulators with incorrect row/column offsets.

10. Edge Cases / Special Cases

- Boundary threads may own a partially valid micro-tile; predicate each store.
- Dynamic indexing into accumulator arrays may prevent scalarization.
- An epilogue can increase live ranges and push a previously safe kernel into spilling.
- Very small K may not amortize extra mapping and initialization work.
- Register allocation occurs at kernel compilation and can vary by architecture/toolchain.
- FP16 inputs commonly use FP32 accumulators, increasing accumulator storage but improving range and accuracy.

11. How to Explain in Interview

Register tiling gives each thread a small output micro-tile. At each K step it loads a short A fragment and B fragment and applies their outer product to multiple register accumulators. That reduces shared-memory accesses and increases ILP, but the tile must be bounded to avoid register spills and harmful occupancy loss.

12. Quick Revision Notes

- One thread owns $T_M \times T_N$ outputs.
- Per K step: $T_M + T_N$ operand loads, $T_M \times T_N$ FMAs.
- Benefit: register reuse plus independent accumulator chains.
- Cost: register pressure, potential spills, more complex mapping.
- Trap: local arrays are not guaranteed to become registers.

13. Practice Tasks

1. Extend a one-output kernel so each thread computes 1×2 , then 2×2 outputs.
2. Count shared loads and FMAs per K step for several micro-tiles.

3. Inspect compiler register counts as tile size grows.
4. Benchmark until performance drops, then check for spills or occupancy changes.
5. Draw which outputs four neighboring lanes store and verify coalescing.
6. Fuse ReLU into the register epilogue and confirm no extra global pass.

14. Final Cheat Sheet

Item	Remember
Core definition	One thread keeps multiple C values in registers.
Why it matters	More operand reuse and ILP at the innermost level.
Most asked	Outer products, register pressure, spills, occupancy.
Main comparison	Shared tiling is inter-thread; register tiling is intra-thread.
One-line answer	"A thread updates a register micro-tile with outer products, trading more reuse for more registers."

Vectorized Loads

1. Overview

Definition. Vectorized loading uses one instruction to move several adjacent scalar elements, such as a 16-byte `float4`, instead of issuing separate scalar load instructions.

In GEMM, vectorized loads can make cooperative global-to-shared transfers more efficient by reducing instruction and address-calculation overhead and matching wide memory transactions. They appear in GEMM kernels, tensor copies, image kernels, and compiler-generated memory pipelines. Interviewers ask about alignment, coalescing, tails, and the important fact that vectorization does not automatically reduce bytes transferred.

2. Core Idea

Think of carrying four bottles in one crate rather than making four hand trips. The same four bottles still travel, but fewer carrying instructions are needed.

```
// Valid only when source and destination meet float4 alignment/layout rules.
float4 v = reinterpret_cast<const float4*>(src)[vector_index];
reinterpret_cast<float4*>(dst)[vector_index] = v;
```

For row-major `A`, a thread can load four consecutive `K` elements when the starting address is 16-byte aligned. Across a warp, consecutive vector indices cover a contiguous region. Step by step: prove alignment; assign non-overlapping chunks; issue wide loads; unpack or store the vector; handle remaining elements with predicates or a scalar tail.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Vector width	Number of adjacent elements per instruction.	<code>float2</code> = 8 bytes, <code>float4</code> = 16 bytes.	Wider is not automatically faster.
Alignment	Address must satisfy the vector type's alignment contract.	<code>float4</code> commonly requires 16-byte alignment.	Base alignment does not guarantee row/subview alignment.
Coalescing	Warp requests should cover contiguous, aligned segments.	Lane 1 loads vector <code>base+41</code> .	Vectorization and coalescing are distinct.
Instruction count	One vector instruction may replace several scalar instructions.	Four scalar loads -> one wide load.	Same byte count, fewer instructions.
Tail handling	Length may not be divisible by vector width.	Vector body plus 0-3 scalar elements.	Prevent out-of-bounds reads.
Layout/stride	Vectorization requires contiguity along the loaded axis.	Row-major A is contiguous in K.	Strided columns cannot be blindly cast.
Aliasing/type rules	C++ access and alignment rules still apply; CUDA vector types are safer when contracts are clear.	Aligned allocation and offsets.	Undefined behavior from arbitrary casts.

4. Real-World Example

A transformer kernel stages FP16 activation tiles. Each lane loads a 16-byte packet containing eight adjacent half values, then writes them into shared memory in the layout required by the compute phase. The wide copy reduces load instructions and produces efficient memory transactions; boundary predicates handle a sequence length that is not a multiple of eight.

5. Diagrams / Mental Models

```

Scalar per lane:  load x0, load x1, load x2, load x3
Vector per lane:  load [x0 x1 x2 x3]

Warp lanes:
lane 0 -> elements  0..3
lane 1 -> elements  4..7
lane 2 -> elements  8..11      contiguous warp footprint
...

Vectorization: fewer instructions
Coalescing: fewer/effective memory transactions across lanes

```

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. What is a vectorized load?	One instruction transfers multiple adjacent scalar values.	Width and contiguity.	Calling multiple thread loads vectorization.
2. Does <code>float4</code> move fewer bytes?	No; it moves the same four floats with fewer load instructions.	Instruction versus byte reduction.	Claiming 4x less bandwidth.
3. Is vectorization the same as coalescing?	No. Vectorization is per-thread width; coalescing combines warp memory requests.	Both levels.	Using the terms interchangeably.
4. Why does alignment matter?	Misaligned vector access may be illegal, compile poorly, or require extra transactions.	Type/address contract.	Checking only allocation base.
5. How do you handle tails?	Predicate the last vector or process remaining scalars separately without over-read.	Correct boundary path.	Reading past allocation and masking afterward.
6. Can columns be vector-loaded in row-major data?	Not directly when elements are strided; change mapping, transpose/layout, or use gathers/scalars.	Physical contiguity.	Mathematical adjacency equals memory adjacency.
7. When does vectorization help?	When scalar instruction/address overhead matters and alignment/coalescing are already sound.	Measure it.	Expecting universal speedup.
8. Can the compiler vectorize automatically?	Sometimes, but explicit types/layouts make guarantees clearer; inspect generated code.	Compiler dependence.	Assuming source syntax proves instruction form.
9. Why use 16-byte packets for FP16?	One packet holds eight halves and commonly matches efficient copy widths.	Bytes, not element name.	Saying <code>half4</code> is always required.
10. What can make it slower?	Misalignment, extra unpack/shuffle work, wasted tail bytes, register pressure, or no instruction bottleneck.	Tradeoff analysis.	Ignoring downstream layout.

7. Deep-Dive Questions

1. **Is an aligned allocation enough?** No. A row start is aligned only if its byte stride preserves alignment; a submatrix column offset can also break it.

2. **How do vectorized loads interact with shared-memory layout?** A wide store is easiest when shared destinations are contiguous and aligned. If compute needs a swizzled layout, unpacking or a designed vector-compatible swizzle may be required.
3. **Why might scalar loads compile into equivalent transactions?** Coalescing operates on warp requests, and caches/memory controllers transfer sectors regardless of source-level types. The difference may be instruction count rather than DRAM traffic.
4. **How should a safe fast path be selected?** Check pointer/stride alignment and divisible extents, use the vector path when valid, and retain a correct scalar/predicated path otherwise.
5. **What profiler evidence matters?** Compare executed global-load instructions, memory sectors/bytes, throughput, and stalls. A speedup with unchanged bytes often comes from reduced instruction overhead.

8. Comparison Tables

Aspect	Scalar loads	Vectorized loads
Values per source instruction	Usually 1	2, 4, or byte-equivalent packet
Alignment requirement	Lower	Stricter
Tail logic	Simple per value	Needs explicit handling
Bytes required by algorithm	Same	Same
Best benefit	Flexibility	Lower instruction/address overhead

9. Common Mistakes

- Believing wide loads reduce the mathematical data volume.
- Casting an unaligned submatrix pointer to `float4*`.
- Reading a full vector past the valid allocation at a boundary.
- Vectorizing along a strided rather than contiguous dimension.
- Ignoring whether the shared-memory destination layout supports wide stores.
- Assuming source-level `float4` guarantees a faster machine instruction.

10. Edge Cases / Special Cases

- A valid row length can still have misaligned subsequent rows if the pitch is unsuitable.
- Predication may operate at whole-vector granularity; mixed-validity vectors need a safe tail path.
- Page/allocation boundaries make speculative over-read unsafe even if masked values are unused.
- Transposed operands may require a different lane mapping.
- Wider per-thread transfers can raise register use or reduce the number of participating loaders.
- Small transfers may see no measurable gain because launch or compute dominates.

11. How to Explain in Interview

Vectorized loads move several adjacent elements per thread instruction, often reducing instruction and address overhead during GEMM tile staging. They complement warp coalescing but are not the same thing, move the same total bytes, and require proven alignment, contiguous layout, and a safe tail path.

12. Quick Revision Notes

- `float4` means four adjacent floats in one typed access, commonly 16 bytes.
- Vectorization is per-thread; coalescing is across a warp.
- Same data volume, potentially fewer instructions.
- Validate base, row, and subview alignment.
- Trap: never over-read a tail merely because invalid lanes are later ignored.

13. Practice Tasks

1. Write scalar and `float4` copy kernels and compare instruction counts.
2. Test aligned and deliberately offset pointers.
3. Add a safe tail for lengths `1..3` beyond a multiple of four.
4. Map warp lanes to vector chunks on paper and verify contiguous coverage.
5. Vectorize A-tile staging while retaining scalar B staging, then measure.
6. Inspect generated assembly to see whether intended wide operations appear.

14. Final Cheat Sheet

Item	Remember
Core definition	One thread instruction moves multiple adjacent values.
Why it matters	Reduces memory-instruction/address overhead.
Most asked	Alignment, coalescing distinction, tails, layouts.
Main comparison	Vector loads reduce instructions, not required bytes.
One-line answer	"Use wide loads for aligned contiguous tile copies, with a correct scalar or predicated tail."

Double Buffering

1. Overview

Definition. Double buffering allocates two staging buffers so the kernel can load the next K tile while computing with the current tile. The buffers alternate roles.

It matters because a fast GEMM must hide memory latency, not only reduce traffic. Modern CUDA kernels combine ping-pong shared memory with asynchronous global-to-shared copies and multi-stage pipelines. Interviewers ask it to test producer-consumer hazards, overlap conditions, synchronization, and resource tradeoffs.

2. Core Idea

Picture two kitchen cutting boards. While the cook uses board 0, an assistant prepares ingredients on board 1. At the end of the step they swap. With one board, preparation and cooking must happen sequentially.

```
prologue: load tile 0 into buffer 0
for each tile t:
    begin loading tile t+1 into buffer 1-current
    compute tile t from buffer current
    wait until tile t+1 is ready
    swap current
```

The true pipeline includes a prologue (fill before first compute), steady state (overlap), and epilogue (finish last compute without another load). Overlap requires hardware/compiler support or asynchronous copy operations; merely writing alternating arrays in sequential code does not guarantee concurrency.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Ping-pong buffers	Two storage regions alternate read/write roles.	<code>As[2][BM][BK]</code> .	Identify current and next stage.
Prologue	Prime the first tile before computation.	Load stage 0, then wait.	Avoid consuming uninitialized data.
Steady state	Load next while computing current.	<code>copy(t+1)</code> overlaps <code>mma(t)</code> .	Where latency is hidden.
Epilogue	Drain remaining compute after final load.	No out-of-range prefetch.	Off-by-one correctness.
Async copy	Allows data movement to progress without tying up ordinary load/store sequences.	CUDA <code>cp.async</code> -style pipeline.	Commit/wait semantics.
Buffer hazard	Producer must not overwrite a buffer still used by consumer.	Wait before reuse.	Correct synchronization.
Stage count	Two is common; deeper pipelines may cover longer latency but cost storage.	2-4 stages.	Double buffering is a special case.

4. Real-World Example

In a large transformer GEMM, each CTA repeatedly consumes K tiles. While Tensor Cores process operands derived from shared-memory stage 0, asynchronous copies populate stage 1 from global memory. This pipeline keeps compute units busy, provided the copy latency fits beneath the tile's compute time and occupancy remains sufficient.

5. Diagrams / Mental Models

```
Time ---->
```



```

Single buffer: [load 0][compute 0][load 1][compute 1][load 2][compute 2]
Double buffer: [load 0]
                  [compute 0]
                  [load 1   ]
                      [compute 1]
                      [load 2   ]

```

```

Stage t: consumer reads buffer t%2
         producer fills buffer (t+1)%2

```

Benefit condition	Consequence
Copy and compute can progress concurrently	Latency can be hidden
Enough computation per tile	Better overlap
Extra shared memory fits comfortably	Residency stays acceptable
Pipeline correctly synchronized	No stale/overwritten data

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. What is double buffering?	Alternating two staging buffers to overlap next-tile loading with current-tile computation.	Producer-consumer pipeline.	Merely using two arrays.
2. Why can it speed up GEMM?	It hides some global-memory latency behind useful arithmetic.	Overlap, not fewer bytes/FLOPs.	Saying it halves memory traffic.
3. What are prologue and epilogue?	Initial fill before overlap and final drain after the last prefetched tile.	Pipeline boundaries.	Applying steady-state indexing everywhere.
4. Why are waits needed?	Consumer must see completed copies and producer must not overwrite live data.	Readiness and reuse hazards.	Synchronizing only once.
5. Does alternating buffers guarantee overlap?	No; operations need asynchronous capability and independent resources/scheduling.	Mechanism versus intent.	Assuming source order overlaps automatically.
6. What is the cost?	Roughly doubled staging storage plus pipeline complexity and possibly lower occupancy.	Resource tradeoff.	Calling buffering free.
7. What if compute is	Some latency remains exposed; use deeper stages, larger compute tile, or	Pipeline balance.	Claiming perfect hiding.

Question	Clear answer	Expected key points	Common mistake
shorter than load latency?	more concurrent warps if resources permit.		
8. What if K has one tile?	No steady-state overlap exists; execute the single load and compute safely.	Boundary case.	Prefetching nonexistent tile.
9. How are edge tiles prefetched?	Use valid-byte/predicate semantics and zero-fill invalid operands.	Correct asynchronous boundary handling.	Out-of-bounds copy.
10. How do you prove it works?	Validate varied K -tile counts and use a timeline/profiler to show copy-compute overlap and reduced stalls.	Correctness plus performance evidence.	Inferring overlap from faster time only.

7. Deep-Dive Questions

- Why use more than two stages?** If memory latency exceeds one tile's compute duration, several prefetched tiles can cover it, at the cost of more shared memory and scheduling complexity.
- What is the relationship between occupancy and pipelining?** More resident warps can hide latency globally; an intra-block pipeline hides it within a CTA. Resource-heavy staging may reduce the former while improving the latter.
- What does commit/wait mean for asynchronous copies?** Copies are grouped or committed, and the consumer waits until the required groups complete before reading their destination. Exact primitives are architecture/API-specific.
- Can register fragments also be double-buffered?** Yes. A kernel may prefetch the next shared-memory fragment into a second register set while issuing compute on the current fragment, increasing register pressure.
- Why is pipeline debugging hard?** An off-by-one stage index or missing wait causes intermittent races that depend on timing. Test one, two, and many K tiles plus partial final tiles.

8. Comparison Tables

Aspect	Single buffering	Double buffering
Stage storage	1x	About 2x
Load/compute schedule	Mostly serial	Potentially overlapped
Control flow	Simpler	Prologue/steady state/epilogue
Hazard surface	Lower	Buffer readiness and reuse
Best use	Low latency or resource-limited kernel	Repeated K tiles with meaningful compute

9. Common Mistakes

- Assuming two buffers automatically create hardware overlap.
- Computing from a stage before its copy completes.
- Overwriting the current stage while consumers still read it.
- Prefetching beyond the final K tile.
- Forgetting zero-fill semantics for a partial final tile.
- Doubling shared memory without checking occupancy impact.
- Benchmarking a one-tile K dimension and expecting a pipeline benefit.

10. Edge Cases / Special Cases

- Zero or one K tile bypasses most pipeline logic.
- A partial last tile needs safe copy size/predication.
- Very small tiles may not contain enough computation to hide copy latency.
- Async-copy alignment and granularity requirements are architecture-specific.
- Deeper stages can exceed shared-memory limits or reduce resident CTAs.
- Synchronization differs when a warp is the complete producer/consumer unit versus a whole block.

11. How to Explain in Interview

Double buffering uses two shared-memory stages: compute reads the current tile while an asynchronous copy fills the next. A prologue primes the pipeline, waits protect readiness and buffer reuse, and an epilogue drains it. It hides latency rather than reducing work, and its extra storage can reduce occupancy.

12. Quick Revision Notes

- Two buffers alternate producer and consumer roles.
- Phases: prologue, overlapped steady state, epilogue.
- Benefit: hidden latency. Cost: storage and synchronization.
- No async mechanism means no guaranteed overlap.
- Trap: handle one-tile and final-partial-tile cases explicitly.

13. Practice Tasks

1. Draw stage ownership for one, two, three, and four K tiles.
2. Add ping-pong buffers to a shared GEMM and validate before enabling async copies.
3. Intentionally remove a wait and run a race-checking tool.
4. Measure shared-memory use and occupancy before and after buffering.
5. Profile whether copy and compute actually overlap.
6. Generalize the stage index from two buffers to S buffers.

14. Final Cheat Sheet

Item	Remember
Core definition	Load next tile while computing current tile using alternating stages.
Why it matters	Hides memory latency behind arithmetic.

Item	Remember
Most asked	Prologue/epilogue, waits, hazards, resource cost.
Main comparison	Single buffer serializes phases; double buffer can overlap them.
One-line answer	"Ping-pong staging pipelines global copies with tile compute, provided readiness and reuse are synchronized."

Tensor Core GEMM

1. Overview

Definition. Tensor Core GEMM uses specialized GPU matrix-multiply-accumulate hardware that computes small matrix fragments at much higher throughput than ordinary scalar CUDA cores for supported data types and shapes.

Tensor Cores power deep-learning training/inference, scientific mixed-precision solvers, and modern GEMM libraries. They matter because peak throughput requires data layout, precision, accumulation, and tile dimensions compatible with hardware matrix instructions. Interviewers ask candidates to separate Tensor Cores from generic GPU cores and to reason about mixed precision, accuracy, and feeding the units efficiently.

2. Core Idea

An ordinary thread-level kernel issues scalar FMAs. A Tensor Core instruction makes a cooperating warp or warpgroup perform a small matrix operation:

$$D_fragment = A_fragment * B_fragment + C_fragment$$

Think of replacing many individual calculators with a dedicated matrix calculator. The calculator is extremely fast, but inputs must be packed in accepted types/layouts, and the rest of the kernel must deliver data quickly enough.

A typical hierarchy is: global matrices -> asynchronously staged shared tiles -> warp fragments -> matrix MMA instruction -> FP32 accumulator fragments -> epilogue/store. The precise fragment shape and API ([WMMA](#), lower-level MMA instructions, newer warpgroup MMA) depend on architecture.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
MMA	Matrix multiply-accumulate instruction on fragments.	$D=A*B+C$.	It is collective, not one scalar operation.

Subtopic	Meaning and why it matters	Example	Common interview angle
Mixed precision	Inputs can be lower precision while accumulation uses a wider type.	FP16/BF16 inputs, FP32 accumulate.	Throughput versus numerical error.
TF32	NVIDIA format/mode using FP32 range with reduced mantissa for Tensor Core multiplication.	FP32-oriented DL acceleration.	Not identical precision to full FP32 multiply.
Fragment shape	Hardware-defined small $m \times n \times k$ operation.	Common APIs expose fixed supported shapes.	Dimensions need padding/tails.
Layout	Row/column-major declarations and shared-memory arrangement must match instruction requirements.	A row-major, B column-major in one API shape.	Wrong layout yields wrong answers.
Accumulators	Distributed registers hold output fragments.	Often FP32.	Lane ownership may be opaque.
Saturating/low-precision types	INT8, FP8, and other formats serve inference/training cases on supporting hardware.	Quantized GEMM.	Scaling and range matter.
Epilogue	Converts, scales, biases, activates, and stores accumulator tiles.	$Y = \text{ReLU}(\alpha * AB + \text{bias})$.	Fusion avoids bandwidth.

4. Real-World Example

A transformer linear layer multiplies FP16 activations by FP16 weights and accumulates into FP32. A tuned library stages large tiles, distributes warp fragments, issues Tensor Core MMA instructions, then applies bias and activation in the epilogue. The application normally calls cuBLAS/cuBLASLt rather than writing MMA assembly because layout, architecture, and shape tuning are complex.

5. Diagrams / Mental Models

```

DRAM -> L2 -> shared-memory CTA tile -> warp fragments -> Tensor Core MMA
                                     |
                                     v
                                register accumulators
                                     |
                                scale/bias/activation
                                     v
                                     C

```

Concern	Question to ask
Compute	Is the operation using a supported MMA type/shape?
Feeding	Are global/shared pipelines fast enough?
Precision	Are input rounding and accumulator range acceptable?
Shape	Are padding and edge paths correct?
Portability	Is there an architecture/library fallback?

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. What is a Tensor Core?	Specialized hardware for small matrix multiply-accumulate operations on supported types.	Matrix instruction and throughput.	Calling it a separate GPU or general CPU core.
2. How does it differ from a CUDA core?	CUDA cores execute ordinary scalar/vector arithmetic; Tensor Cores execute collective matrix operations.	Specialized versus general datapath.	Saying Tensor Cores replace all CUDA cores.
3. Why mixed precision?	Lower-precision inputs raise throughput and reduce bandwidth; wider accumulation preserves more accuracy/range.	Performance-accuracy compromise.	Saying FP32 accumulation restores lost input bits.
4. What is WMMA?	A CUDA warp-level API for loading fragments, performing MMA, and storing results on supported hardware.	API abstraction and warp cooperation.	Treating each thread's fragment as a normal matrix.
5. Must dimensions match fragment multiples?	Fast paths generally tile/pad to supported shapes and use edge/fallback handling for leftovers.	Hardware shape constraints.	Ignoring tails.
6. Why might Tensor Core GEMM be slow?	Small size, poor layout, conversion/padding, insufficient staging, low occupancy, or memory/epilogue bottlenecks.	Whole pipeline matters.	Equating peak TOPS with application speed.
7. What is TF32?	A Tensor Core input mode with FP32 exponent range but reduced mantissa precision, typically accumulating in FP32.	Range versus precision.	Calling it ordinary IEEE FP32 multiplication.

Question	Clear answer	Expected key points	Common mistake
8. Are Tensor Core results bit-identical to FP32 GEMM?	Generally no; input formats, rounding, FMA order, and algorithm choices differ.	Numerical semantics.	Exact equality expectation.
9. When should an application use a library?	Almost always for standard GEMM; libraries select tuned kernels and epilogues across shapes/hardware.	cuBLAS/cuBLASLt.	Hand-writing assembly first.
10. How do you verify Tensor Core use?	Inspect profiler instruction/pipe metrics or generated machine code, and confirm math-mode/type configuration.	Evidence, not timing alone.	Assuming FP16 automatically uses Tensor Cores.

7. Deep-Dive Questions

- Why does FP32 accumulation not make FP16 input exact?** Values are rounded when represented or multiplied at lower precision; the accumulator cannot recover discarded information, though it reduces subsequent accumulation error.
- How can loss scaling help training?** Scaling gradients upward before low-precision computation prevents small values from underflowing; unscaling and overflow checks preserve usable updates.
- Why are shared-memory layouts often swizzled?** MMA lane access patterns can otherwise create severe bank conflicts; swizzles distribute addresses across banks while retaining logical fragment layout.
- What is the roofline implication?** Tensor Cores greatly raise the compute roof, so feeding them requires high reuse; a formerly compute-bound operation can become limited by memory, instructions, or epilogue work.
- What is split-K with Tensor Cores?** Several CTAs process disjoint K ranges for the same output tile to increase parallelism, then combine partial accumulators, paying reduction traffic and synchronization.

8. Comparison Tables

Aspect	CUDA-core FP32 GEMM	Tensor Core GEMM
Primitive	Scalar/thread FMA	Warp/warpgroup matrix MMA
Types	Flexible ordinary arithmetic	Supported matrix input/accumulator types
Peak throughput	Lower for GEMM	Much higher on supported hardware
Layout constraints	Moderate	Stronger fragment/layout constraints
Numerical behavior	Full FP32 path possible	Often mixed/reduced-precision multiply
Best implementation	Tuned kernel/library	Architecture-tuned library

9. Common Mistakes

- Assuming any FP16 multiplication automatically uses Tensor Cores.
- Treating fragments as independently owned ordinary thread arrays.

- Claiming FP32 accumulation removes all low-precision error.
- Ignoring supported shapes, alignment, layouts, and architecture capability.
- Quoting theoretical peak without considering memory and epilogue bottlenecks.
- Writing a custom kernel when a library already provides a tuned operation.
- Validating with exact equality across different math modes.

10. Edge Cases / Special Cases

- Unsupported GPUs require a fallback path.
- Odd dimensions need padding, predication, or a non-MMA remainder kernel.
- NaNs, infinities, denormals, saturation, and overflow differ across formats/modes.
- Very small matrices may not amortize launch, packing, and fragment setup.
- Quantized integer/FP8 GEMM requires correct scales, zero points where applicable, and accumulator range.
- Deterministic or strict-precision requirements may forbid faster algorithms/math modes.

11. How to Explain in Interview

Tensor Cores execute warp-level matrix multiply-accumulate instructions on fixed fragments, often using low-precision inputs and wider accumulators. High performance still needs hierarchical tiling and a fast memory pipeline, while production code must handle layout, tail dimensions, architecture support, and numerical tolerance.

12. Quick Revision Notes

- Tensor Core primitive: fragment MMA, not scalar FMA.
- Common pattern: low-precision input, FP32 accumulator.
- WMMA is an API; exact instructions/shapes depend on architecture.
- Peak compute requires aggressive data reuse and staging.
- Trap: wider accumulation cannot restore precision lost in inputs.

13. Practice Tasks

1. Implement a small WMMA GEMM for supported clean dimensions.
2. Compare FP16/FP32-accumulate output against an FP64 CPU reference.
3. Add padding for non-fragment-multiple dimensions and crop valid output.
4. Profile to verify Tensor Core instructions are active.
5. Compare a custom implementation with cuBLAS on small and large shapes.
6. Test an ill-conditioned matrix and explain error changes between math modes.

14. Final Cheat Sheet

Item	Remember
Core definition	Specialized hardware performs small matrix MMA collectively.
Why it matters	Very high GEMM throughput on supported types/shapes.
Most asked	Mixed precision, WMMA, layouts, tails, verification.

Item	Remember
Main comparison	Tensor Cores do matrix instructions; CUDA cores do general arithmetic.
One-line answer	"Tensor Core GEMM maps tiled fragments to specialized MMA hardware and feeds it through a tuned memory pipeline."

Warp-Level Matrix Operations

1. Overview

Definition. Warp-level matrix operations are matrix loads, multiply-accumulates, data exchanges, and stores performed cooperatively by the lanes of a warp (or, on newer mechanisms, a warpgroup). No single lane necessarily owns a recognizable full input fragment.

They are the execution layer connecting shared-memory tiles to Tensor Core or SIMD-style matrix computation. They appear in WMMA, architecture-specific MMA instructions, CUTLASS-style kernels, and custom fused operators. Interviewers ask about warp cooperation, divergence, fragment ownership, synchronization, and the hierarchy from block tiles to warp tiles.

2. Core Idea

A warp is a team of lanes executing one collective matrix instruction. The fragment is distributed across lane registers according to a hardware/API-defined mapping. Lanes must participate consistently; thinking of the fragment as 32 separate tiny conventional matrices is usually wrong.

```
// Conceptual WMMA flow; exact types/shapes depend on the API and GPU.
fragment<matrix_a, M, N, K, half, row_major> a;
fragment<matrix_b, M, N, K, half, col_major> b;
fragment<accumulator, M, N, K, float> c;
fill_fragment(c, 0.0f);
load_matrix_sync(a, A_tile, lda);
load_matrix_sync(b, B_tile, ldb);
mma_sync(c, a, b, c);
store_matrix_sync(C_tile, c, ldc, mem_row_major);
```

Step by step: assign each warp an output warp tile; load compatible operand fragments; issue collective MMA operations across K fragments; retain distributed accumulators in registers; apply an epilogue or store collectively.

3. Important Subtopics

Subtopic	Meaning and why it matters	Example	Common interview angle
Warp tile	Output region assigned to one warp.	Several warp tiles form a CTA tile.	Hierarchical ownership.
Fragment	Distributed register representation consumed by a matrix instruction.	A/B/accumulator fragments.	Layout may be opaque.
Collective participation	Required lanes execute compatible matrix operations together.	Uniform <code>mma_sync</code> .	Divergence can invalidate behavior.
Warp shuffle	Direct register exchange among lanes for some custom microkernels/reductions.	<code>__shfl_sync</code> .	Avoid shared memory for warp-local exchange.
Lane mapping	Defines which logical elements each lane loads/owns.	Architecture/API-specific.	Do not assume linear ownership.
Warp synchronization	<code>__syncwarp(mask)</code> orders participating lanes when needed.	Producer-consumer via shared memory.	Lockstep is not a general memory barrier.
Warpgroup operations	Newer architectures may use multiple warps for larger asynchronous matrix operations.	Warpgroup MMA.	Capability-specific execution model.

4. Real-World Example

A fused attention kernel computes score and value products without materializing every intermediate. Each block owns a query/key tile, each warp owns smaller matrix regions, and warp-level matrix instructions update register fragments. Warp shuffles or shared memory help compute row reductions for softmax, after which matrix operations continue. This reduces global traffic but demands careful synchronization and numerical handling.

5. Diagrams / Mental Models

CTA output tile

```

+-----+
| Warp 0 tile | Warp 1 tile |
|-----+-----|
| Warp 2 tile | Warp 3 tile |
+-----+

```

Inside Warp 0:

```

lane registers collectively hold A fragment, B fragment, C fragment
32 lanes -- one logical MMA operation --> updated C fragment

```

Do not picture: lane 0 owns matrix row 0, lane 1 owns row 1.
The mapping is instruction/API-defined.

6. Common Interview Questions

Question	Clear answer	Expected key points	Common mistake
1. What is a warp-level matrix operation?	A matrix operation whose operands/results are distributed across cooperating warp lanes.	Collective execution.	One thread performs the whole fragment.
2. What is a fragment?	An API/hardware-defined distributed register container for a matrix tile.	Logical tile, distributed storage.	Indexing it like a portable row-major array.
3. Why must lanes agree?	Collective instructions expect participating lanes to execute compatible operations with valid operands.	Uniform control flow.	Allowing arbitrary divergence around MMA.
4. What is a warp tile?	The output subregion assigned to one warp within a block tile.	CTA-to-warp hierarchy.	Confusing it with one instruction fragment.
5. Warp shuffle versus shared memory?	Shuffles exchange registers within a warp; shared memory supports broader/block communication and arbitrary staging.	Scope and use.	Assuming shuffles exchange with other warps.
6. Is warp execution always lockstep?	Lanes issue together, but independent thread scheduling means programs must use correct masks/synchronization for dependencies.	Modern correctness model.	Relying on implicit lockstep memory ordering.
7. How are multiple K fragments handled?	Repeated collective MMA calls accumulate into the same register fragment.	Reduction loop.	Storing after every fragment.
8. Can a warp compute several instruction tiles?	Yes; a warp tile can contain multiple MMA fragments in M/N and iterate across K.	Warp tile versus instruction tile.	Treating the sizes as identical.
9. How are fragments stored to global memory?	Use supported collective store operations or an epilogue/layout transformation with valid leading dimension.	Layout-aware store.	Assuming per-lane contiguous output.
10. What is a warpgroup?	A cooperating set of multiple warps used by some newer architecture-level matrix mechanisms.	Architecture dependence.	Applying warpgroup semantics to every GPU.

7. Deep-Dive Questions

1. **Why is manual fragment element access risky?** The lane-to-element mapping can be undocumented, API-specific, or architecture-dependent. Use defined load/MMA/store operations unless the API explicitly supports uniform element transforms.
2. **How do warp tiles reduce synchronization cost?** Warp-local register exchange and collective operations avoid block barriers when data never crosses warp boundaries, though block synchronization remains necessary for shared tiles produced by the full CTA.
3. **How is a CTA tile decomposed?** Choose a warp grid such as $\text{warpsM} \times \text{warpsN}$; each warp accumulates its output tile, and each warp tile comprises one or more instruction fragments.
4. **What masks should shuffles use?** A mask representing exactly the active participating lanes. Using a full mask when some named lanes do not participate is incorrect.
5. **Why can an epilogue use shared memory?** Distributed accumulator ownership may not match coalesced global-store order. A shared-memory rearrangement can convert the compute layout into an efficient output layout and support fusion.

8. Comparison Tables

Aspect	Thread-level scalar operation	Warp-level matrix operation	Block-level tiled GEMM
Cooperation scope	One thread	One warp/warpgroup	Multiple warps in a block
Typical storage	Thread registers	Distributed fragment registers	Shared memory plus registers
Synchronization	None for private data	Uniform collective operation; warp sync as needed	Block barriers for shared stages
Output unit	Scalar/micro-tile	Warp tile/instruction fragments	CTA tile
Primary purpose	Flexible arithmetic	High-throughput matrix compute	Data reuse and orchestration

9. Common Mistakes

- Treating a fragment as an ordinary per-thread matrix.
- Diverging lanes around a collective matrix instruction.
- Assuming warp tile equals hardware instruction tile.
- Using shuffles to communicate across warps.
- Relying on implicit lockstep instead of correct masks/synchronization.
- Hard-coding a lane mapping that is not guaranteed by the API.
- Ignoring the mismatch between accumulator and coalesced store layouts.

10. Edge Cases / Special Cases

- Partial warps and divergent branches require valid participation masks for supported warp primitives.
- Fragment dimensions and operand layouts vary by architecture and data type.
- Shared-memory producer data may still require a block barrier before a consumer warp loads fragments.
- Edge matrix tiles may need padding because collective loads often assume complete fragments.

- Warpgroup operations have different synchronization and lifetime rules from classic warp MMA.
- A fused reduction such as softmax introduces cross-lane numerical and masking edge cases.

11. How to Explain in Interview

Warp-level matrix operations let a warp collectively load distributed fragments, execute matrix MMA instructions, and retain a warp tile in register accumulators. The lane layout is API-defined, so control flow must be uniform and stores must respect the fragment layout. They sit inside block-level tiling, which supplies reusable shared-memory tiles.

12. Quick Revision Notes

- Warp lanes collectively own fragments; no lane necessarily owns a logical row.
- CTA tile > warp tile > instruction fragment.
- Repeat MMA across K into distributed accumulators.
- Shuffles are warp-local; shared memory connects warps.
- Trap: modern correctness cannot rely on informal warp lockstep.

13. Practice Tasks

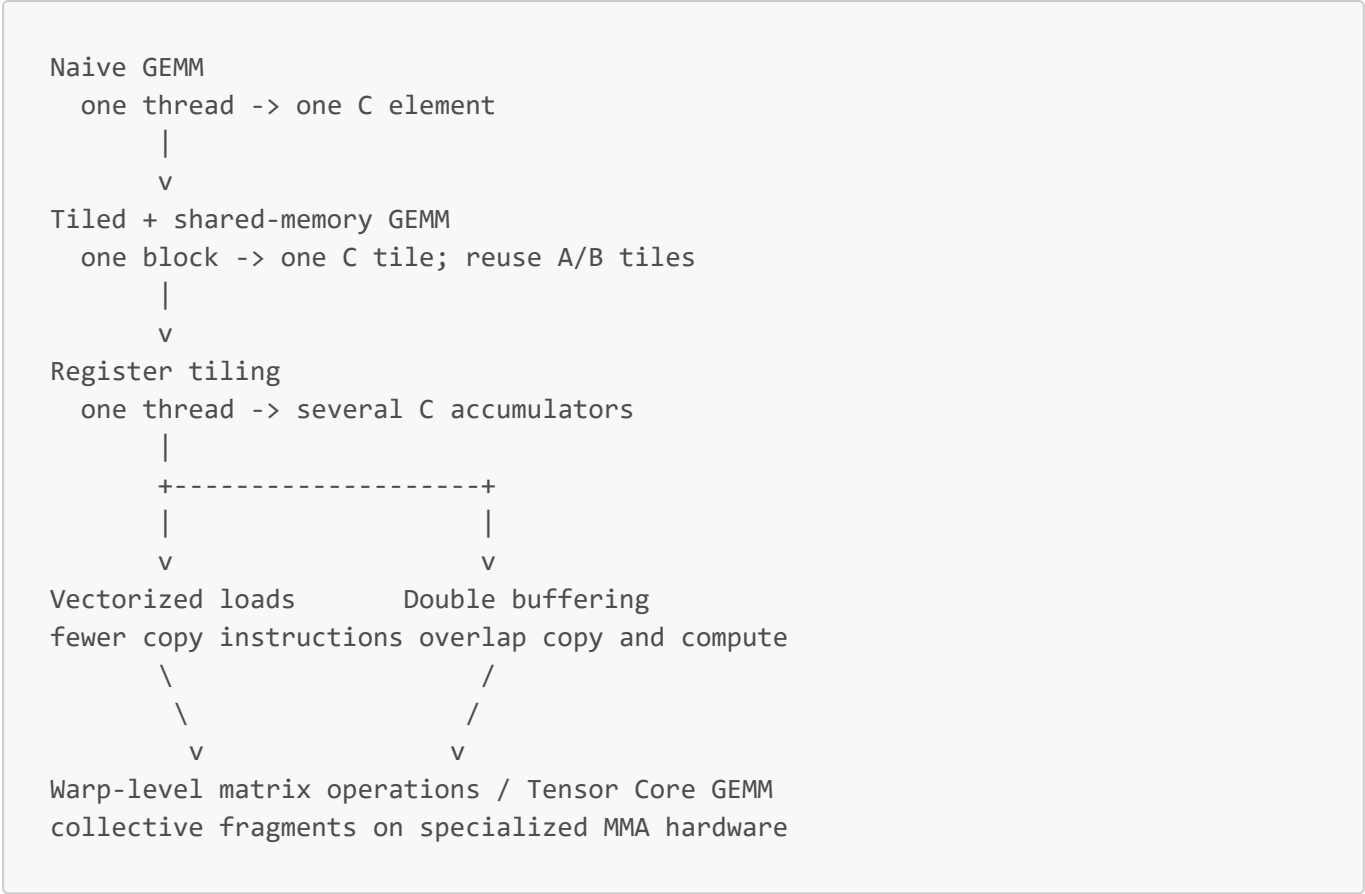
1. Draw a CTA with four warps and assign each a distinct output tile.
2. Implement a supported WMMA load-MMA-store example.
3. Add several K-fragment iterations and verify accumulation.
4. Use warp shuffles to reduce 32 values and explain the active mask.
5. Compare a direct fragment store with a shared-memory epilogue conceptually.
6. Profile warp-level matrix instruction utilization and stall reasons.

14. Final Cheat Sheet

Item	Remember
Core definition	Warp lanes cooperate on distributed matrix fragments.
Why it matters	Maps warp tiles to high-throughput matrix instructions.
Most asked	Fragment ownership, uniformity, warp tiles, sync, shuffles.
Main comparison	Warp operations compute; block tiling stages and coordinates.
One-line answer	"A warp collectively transforms distributed operand fragments into register accumulator fragments using MMA."

End-to-End GEMM Optimization Map

The techniques are cumulative rather than eight unrelated alternatives:



Optimization	Primary bottleneck addressed	Main new risk
Tiling	Redundant global traffic	Poor tile choice
Shared memory	Lack of explicit block reuse	Barriers, bank conflicts
Register tiling	Shared traffic and FMA latency	Spills, low occupancy
Vectorized loads	Copy instruction overhead	Alignment and tails
Double buffering	Exposed memory latency	Pipeline races/resources
Warp-level MMA	Scalar instruction throughput	Collective/layout constraints
Tensor Cores	Matrix compute throughput	Precision and feeding the units

For a placement interview, begin with the mathematical mapping, identify data reuse, then move through the memory hierarchy. State correctness requirements before tuning: valid dimensions, predicated edges, uniform barriers, compatible layouts, and tolerance-based numerical validation. In real application code, prefer a tuned GEMM library unless fusion or a specialized shape justifies a custom kernel.